

2-mavzu. Python dasturlash tili alfaviti

Reja.

1. Python tili sintaksisi, asosiy operatorlari,
2. Bir nechta maxsus holatlar, O'zgaruvchilar, Operatorlar va ifodalar.
3. Python dasturlash tilida arifmetik operatorlar. O'zlashtirish operatorlari.

1. Python tili sintaksisi, asosiy operatorlari.

Arifmetik amallar va qiymat berish operatori. Berilganlarni qayta ishlash uchun PYTHON tilida amallarning juda keng majmuasi aniqlangan. Amal - bu qandaydir harakat bo'lib, u bitta (unar) yoki ikkita (binar) operandlar ustida bajariladi, hisob natijasi uning qaytariluvchi qiymati hisoblanadi. Tayanch arifmetik amallarga qo'shish (+), ayirish (-), ko'paytirish (*), bo'lish (/), darajaga ko'tarish (**) va bo'lish qoldig'ini olish (%) amallarini keltirish mumkin. Amallar qaytaradigan qiymatlarni o'zlashtirish uchun qiymat berish amali (=) va uning turli modifikatsiyalari ishlatiladi: qo'shish, qiymat berish bilan (+); ayirish, qiymat berish bilan (-); ko'paytirish qiymat berish bilan (*); bo'lish, qiymat berish bilan (/); bo'lish qoldig'ini olish, qiymat berish bilan (%) va boshqalar. Ularning umumiy ko'rinishlariga to'xtalamiz. Razryadli mantiqiy amallar. Dastur tuzish tajribasi shuni ko'rsatadiki, odatda qo'yilgan masalani yechishda biror holat ro'y berganligini yoki yo'qligini ifodalash uchun 0 va 1 qiymat qabul qiluvchi bayroqlardan foydalaniladi. Shu maqsadda bir yoki undan ortiq baytli o'zgaruvchilardan foydalanish mumkin. Masalan, bool (mantiqiy) tupdagi o'zgaruvchini shu maqsadda ishlatib bo'ladi. Boshqa tomondan, bayroq sifatida baytning razryadlaridan foydalanish ham mumkin. Chunki razryadlar faqat ikkita qiymatni – 0 va 1 sonlarini qabul qiladi. Bir baytda 8 razryad bo'lgani uchun unda 8 ta bayroqni kodlash imkoniyati

Arifmetik operatorlar

Biz bilgan matematik amallarni bajarish uchun arifmetik operatorlardan foydalaniladi.

```
void main() {  
int x = 6, y = 8;  
print(y / x); // bu kabi bo'lishda natija butun va kasr qismdan iborat  
print(y ~/ x); // bu kabi bo'lishda esa faqat butun qism olinadi  
print(y % x); // qoldiq bo'lish ya'ni qoldiq topiladi  
}
```

```
>> 1.3333333333333333  
>> 1  
>> 2
```

Qolgan barcha arifmetik operatorlar ishlashi bilan o'zingiz bir tanishib chiqasiz.

Tayinlash operatorlari

O'zgaruvchilarga qiymat berish uchun tayinlash operatorlari foydalaniladi. Masalan, += qiymatni oshirish operatoridan foydalanib o'zgaruvchini qiymatini oshirishni bir misol orqali ko'rib o'taylik :

```
void main() {
int number = 8;
number += 5;
print(number);
}
```

>> 13

7.2. Bir nechta maxsus holatlar, O'zgaruvchilar, Operatorlar va ifodalar.

Razryadli mantiqiy amallar. Dastur tuzish tajribasi shuni ko'rsatadiki, odatda qo'yilgan masalani yechishda biror holat ro'y berganligini yoki yo'qligini ifodalash uchun 0 va 1 qiymat qabul qiluvchi bayroqlardan foydalaniladi. Shu maqsadda bir yoki undan ortiq baytli o'zgaruvchilardan foydalanish mumkin. Masalan, bool (mantiqiy) tupdagi o'zgaruvchini shu maqsadda ishlatsa bo'ladi. Boshqa tomondan, bayroq sifatida baytning razryadlaridan foydalanish ham mumkin. Chunki razryadlar faqat ikkita qiymatni – 0 va 1 sonlarini qabul qiladi. Bir baytda 8 razryad bo'lgani uchun unda 8 ta bayroqni kodlash imkoniyati mavjud. Quyidagi jadvalda Python tilida bayt razryadlari ustida mantiqiy amallar majmuasi keltirilgan.

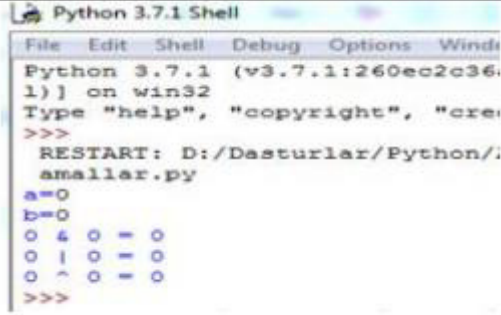
Bayt razryadlari ustida mantiqiy amalla

Amallar	Mazmuni
And yoki &	Mantiqiy VA (ko'paytirish)
Xor yoki 	Mantiqiy yoki (qo'shish))
Or yoki ^	Istisno qiluvchi YOKI

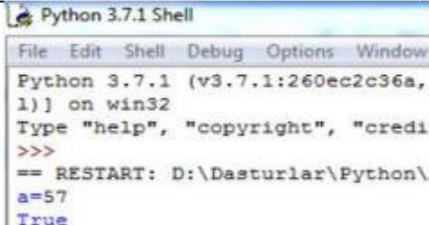
Razryadli mantiqiy amallarning bajarish natijalarini jadval ko'rinishida ko'rsatish mumkin.

A	B	A&B	A B	A^B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Razryadli mantiqiy amallarni bajarish natijalari

<pre>a=int(input('a=')) b=int(input('b=')) print(a,'&',b,'=',a&b) print(a,' ',b,'=',a b) print(a,'^',b,'=',a^b)</pre>	
---	--

1.1-masala. A butun soni berilgan. Jumlani rostlikka tekshiring: “A soni toq son”.

<pre>a=int(input('a=')) print(bool(a%2==1))</pre>	
---	--

Operator – bu ma’lum bir vazifa bajarishga qaratilgan buyruq.

Taqqoslash operatorlari

Taqqoslash operatorlari ikkita qiymatni yoki o‘zgaruvchilarni solishtirish uchun ishlatiladi. Taqqoslashda qaytariladigan qiymat 0 yoki 1, True (1) va False (0) degan ma’noni anglatadi. Bu biz ishlashimiz uchun juda foydali. Chunki kompyuterga berilgan buyruqlar shu tarzda ishlaydi.

```
void main() {
int x = 6, y = 8;
print("x == y | ${x == y}");
}
```

```
>> x == y | False
```

Mantiqiy operatorlar

Mantiqiy operatorlar yordamida rost yoki yolg‘on qiymatlarni tekshiringiz mumkin. Mantiqiy operatorlar o‘zgaruvchilar yoki qiymatlar orasidagi mantiqni aniqlash uchun ishlatiladi. Agarda matematika fanida siz dizyunksiya, konyunksiya va inkor mantiqiy amallari bilan ishlashni o‘rgangan bo‘lsangiz umuman qiynalmaysiz foydalanishda. Bu operatorlar qo‘llanilganda natija doim **True** yoki **False** bo‘ladi.

```
void main() {
int x = 6, y = 8;
var and = x < 5 && y < 10;
print(anE);

var not = !(x < 5 && y < 10);
print(not);
}
```

```
}  
>> False  
>> True
```

7.3. Python dasturlash tilida arifmetik operatorlar. O'zlashtirish operatorlari.

Murakkab shartli ifodalarni yozish, odatda mantiqiy amallar yordamida amalga oshiriladi. Pythonda quyidagi mantiqiy operatorlar mavjud:

`and` (mantiqiy ko'paytirish). Murakkab ifodadagi biror bir qism ifodani qiymati `False` bo'lsa, ifodaning yakuniy qiymati `False`, aks holda `True` qiymat qaytaradi. Masalan:

Copy

```
yoshi = 21  
vazni = 72  
natija = yoshi > 17 and vazni == 72  
print(natija) # True
```

Yuqoridagi dasturda murakkab mantiqiy amal ikki qismdan `yoshi > 17` va `vazni > 56` qismlardan tashkil topgan bo'lib, ular `and` mantiqiy operatori bilan birlashtirilgan. Agarda ikkala mantiqiy amal `True` qiymat qaytarsa ifodaning qiymati `True` bo'ladi, aks holda `False` qiymat qaytaradi.

Mantiqiy ifodalarda faqatgina taqqoslash amallaridan foydalanish shart emas. Ixtiyoriy mantiqiy amal yoki *boolean* turidagi qiymatlar (`True`, `False`) ham ishlatilishi mumkin. Masalan:

Copy

```
yoshi = 21  
vazni = 72 t = True  
natija = yoshi > 17 and vazni > 56 and t  
print(natija) # True
```

`or` (mantiqiy qo'shish). Agarda ifodadagi biror bir qism ifoda `True` qiymat qaytarsa, yakuniy natija ham `True`, aks holda `False` bo'ladi.

Copy

```
yoshi = 21 t = False  
natija = yoshi > 17 or t  
print(natija) # True
```

`not` (mantiqiy inkor). Ifodaning qiymatini `True` bo'lsa, natija `False` va aksincha.

Copy

```
yoshi = 21 t = False  
print(not yoshi > 17) # False  
print(not t) # True
```

`and` operatorining biror bir operandi `False` qiymatga ega bo'lsa, u holda boshqa operand qiymati tekshirib (hisoblaniB) o'tirilmaydi, har doim natija `False` bo'ladi. Bunday xususiyat ish unumdorligini bir oz bo'lsada oshirish imkonini beradi. Xuddi shunaqa xususiyat `or` operatori uchun ham o'rinli. Ya'ni `or` operatorining biror bir operandi qiymati `True` qiymatga ega bo'lsa, boshqa operandlar tekshirilmaydi, natija sifatida har doim `True` qiymati qaytariladi.

Agar bitta ifodada bir nechta mantiqiy operatorlar qatnashgan bo'lsa, u holda ularning ustunligiga (prioritetiga) alohida e'tibor qatarish kerak. Dastlab *not* operatori keyin *and* va eng so'ngra *or* operatori bajariladi. Masalan:

Copy

```
yoshi = 22 xolati = False vazni = 58
natija = vazni == 58 or xolati and not yoshi > 21 # True
print(natija)
```

Ushbu dasturda keltirilgan ifodadagi mantiqiy amallar quyidagi ketma- ketlikda bajariladi:

1. *not yoshi > 21* mantiqiy ifoda *False* qiymat qaytaradi;
2. *xolati and False (not yoshi > 21)* esa *False* qiymat qaytaradi;
3. *vazni == 58 or True (xolati and not yoshi > 21)* esa *True* qiymat qaytaradi.

Shuni alohida ta'kidlash kerarki, mantiqiy ifodalarda mantiqiy amallarning bajarilish ketma-ketligini qavslar (,) yordamida o'zgartirish mumkin.

Operatorlar o'zgaruvchilar va qiymatlar ustida turli amallarni bajarish uchun ishlatiladi.

Bitli operatorlar

Sonlar ustida ikkilik sanoq sistemasi asnosida amallar bajarishda bitli operatorlardan foydalaniladi. Bitli operatorlar oddiy operatorlarga nisbatan ishlashi ~66% ga tezroqdir. Bitli operatorlar ishlashi Bul algebrasida asoslanadi.

Bitli va (&) operatori ishlashini qisqacha ko'rib o'tsak,

2 & 3 = 2, bunda sonlar avval ikkilik sanoq sistemasiga o'tkazib olinadi. 10 & 11 qilib keyin operator o'z vazifasini bajaradi. & – konyunksiya bo'lib, 1 & 1 = 1, 1 & 0 = 0 natija 10 bo'ladi va uni o'nlikka o'tkazilganda 2 chiqadi.

Operatorlar tuzilishiga ko'ra 2 ta turga bo'linadi : unar va binar.

Unar operatorlar bitta operanddan, binar operatorlar esa ikkita operanddan tashkil topadi.

x & y – misolini olsak, bunda x va y operandlar hisoblanadi. & esa operator va uning ikki tomonida ham operand borligi uchun binar operatorga kiradi. Unar operatorlarda chap tomonida hech qanday operand bo'lmaydi. Masalan, !x ifodasi.

Inkrement va dekrement amallari

O'zgaruvchi qiymatini o'zgartirish uchun ishlatilovchi amallar hisoblanadi. Hamda bular 2 turga bo'linadi : postfiks (keyin) va prefiks (oldin).

Bu amallar ishlashdagi farqlar qandayligini ko'rib ketsak,

```
// Postfiks inkrement
void main() {
var x = 5;
var y = x++;
print("$x $y");
}
```

```
>> 6 5
```

Birinchi x qiymati y ga o'zlashtiriladi va keyin x ning qiymati 1 ga ortadi.

```
// Prefiks inkrement
```

```
void main() {  
var x = 5;  
var y = ++x;  
print("$x $y");  
}
```

```
>> 6 6
```

Birinchi x ning qiymati 1 ga ortadi va keyin x qiymati y ga o‘zlashtiriladi.

```
// Postfiks dekrement  
void main() {  
var x = 5;  
var y = x--;  
print("$x $y");  
}
```

```
>> 4 5
```

Birinchi x qiymati y ga o‘zlashtiriladi va x ning qiymati 1 ga kamayadi.

```
// Prefiks dekrement  
void main() {  
var x = 5;  
var y = --x;  
print("$x $y");  
}
```

```
>> 4 4
```

Birinchi x ning qiymati 1 ga kamayadi va keyin x qiymati y ga o‘zlashtiriladi.